

Internet-GIS WS22/23, Task 5:

In this week's exercise, we have a look at the data you will later display as part of your web map application. The data is provided to you as tables in a PostGIS database, hosted on a server that is available only from LUH LAN or through LUH VPN (see <https://www.luis.uni-hannover.de/en/services/communication/netz/services/kommunikation/netz/angebote-im-service-netz/vpn-service>). As the interface to the data requires knowledge about php programming (see Lecture 7 on Nov 29th), you will for now access the data directly using SQL.

Host/IP : gis.ikg.uni-hannover.de / 130.75.188.102
DB name : webgis
Username : stud
Password : stud

You do not have to upload anything to Stud.IP for this exercise. However, use the time on this exercise to develop (and store) the SQL queries, as you will use them inside your php code to access the data from the database. If you participate in the final exercise, send your results for **Task 5.5** in a mail to colin.fischer@ikg.uni-hannover.de.

Task 5.1: Connect to Database

1. Launch the Windows Command Line (Console) cmd.exe. (Open the Windows Start menu and type "cmd" followed by Enter key.)
2. Inside the console window, run a PostGIS client (e.g. psql), e.g. by typing:
`psql -h gis.ikg.uni-hannover.de -U stud webgis`
from the directory containing psql.exe. Alternatively, use the psql files uploaded with the lecture materials. You can directly execute the .bat file contained in the .zip.
3. You should now be inside the psql SQL console (lines should be prefixed with „webgis=>“)
4. Execute the following psql-commands:
`\dt` (list all tables in the database)
`\q` (exit the program)

Task 5.2: Explore the database using SQL

1. In the following examples, feel free to exchange table names and filter criteria depending on the topic of your final exercise.
2. Display the header of a table (without any selected lines). SQL is not case sensitive, capital letters only used for clarity. The clause "where false" makes sure no actual data lines are selected. The result is an empty table having all columns. Please note the required semicolon to finish every SQL command.

```
SELECT * FROM planet_osm_point WHERE false;
```

3. Please note: when executing any SQL commands here, the program will spend 99% of the run-time on formatting the textual output of the table. Access to the data will be much faster when accessing the data from your code. The following command will probably take very long to execute, as the data tables contain millions of lines of data.

```
SELECT * FROM planet_osm_point;
```

4. Now filter the table depending on the filter expression in the final exercise topics table. For the topic “shops” you find the expression “shop =*”, which means: the value of column “shop” is not empty, or in SQL syntax. Feel free to adapt this filter to your group’s topic (see also Task 5.5):

```
SELECT * FROM planet_osm_point WHERE shop IS NOT NULL;
```

5. You can aggregate/count results using:

```
SELECT count(*) FROM planet_osm_point WHERE shop IS NOT NULL;
```

6. Now have a look at the entries in the column “shop” (again: adapt to your group’s topic). You can simply output the attribute “shop” for all lines:

```
SELECT shop FROM planet_osm_point WHERE shop IS NOT NULL;
```

7. Aggregate and count the values in this column. ORDER BY <column> DESC orders results in descending order. From this, you can see the relevance/distribution of the values in the overall data set:

```
SELECT shop, count(*) FROM planet_osm_point WHERE shop IS NOT NULL GROUP BY shop ORDER BY count(*) DESC;
```

8. Start thinking about which categories among the all available categories you want to use in your web map (e.g. top X most relevant/most common). Inspect the other columns (except the geometry column “way”) in the table: which non-spatial attributes might also be relevant in the context of your topic? If a column is mostly empty for the selection based on your group’s topic, this attribute is probably not related to your topic.

```
SELECT name FROM planet_osm_point WHERE shop IS NOT NULL;
```

Task 5.3: Geometry column access

1. Inspect the content of geometry column ‘way’. Because the formatting of the table takes very long, try to formulate a query resulting in a single object (e.g. using the unique field ‘osm_id’). You should see the geometry in WKB format.

```
SELECT osm_id FROM planet_osm_point;  
SELECT way FROM planet_osm_point WHERE osm_id='<one of the above>';
```

2. Now start using PostGIS functions inside the SQL query, for example to modify the output of the previous query. The following output formats should be suitable for parsing using JavaScript inside the client-side part of your application.

```
SELECT ST_AsText(way) FROM planet_osm_point WHERE ...;  
SELECT ST_AsEWKT(way) FROM planet_osm_point WHERE ...;  
SELECT ST_AsGeoJSON(way) FROM planet_osm_point WHERE ...;
```

3. Mind the different projections used by the Leaflet web map (4326) and by the PostGIS DB (3857). Transform coordinates as part of the SQL query.

```
SELECT ST_AsEWKT(ST_Transform(way, 4326)) FROM planet_osm_point WHERE ...;
```

4. Retrieve all objects from a bounding box with given coordinates. Select suitable WGS84 coordinates (e.g. using a web application) and transform to SRID 3857 OR look at the output from the previous stop. Build a geometry from these coordinates on the fly. Use this geometry to select all features within this bounding box.

```
SELECT ST_AsEWKT(ST_GeomFromText('LINESTRING(xmin ymin, xmax ymax)',  
3857));  
SELECT ST_AsEWKT(ST_Transform(ST_GeomFromText('LINESTRING(xmin ymin,  
xmax ymax)', 4326), 3857));
```

```
SELECT * from planet_osm_point WHERE shop IS NOT NULL AND way && ST_Geom-  
FromText('LINESTRING(xmin ymin, xmax ymax)', 3857);
```

Task 5.4: (Optional) Inspecting data using QGIS

1. The free GIS QGIS has a nice interface to import PostGIS data as map layers. This step is not mandatory for the final exercise, but it might give you an idea what the data inside the DB looks like in a map. The following manual is probably too confusing to follow, as the Program was, unfortunately, installed in German language. We can do this together during next week's exercise slot (Monday, Dec 7th).
2. Download, install and execute the Program QGIS.
3. Add a PostGIS-Layer ("layer > add layer" > "add PostGIS layer"). In the settings dialogue, make the following settings:

Window "Data source management | PostgreSQL" (or similar)

click Button "New"

Window „Create new PostgreSQL connection“ (or similar)

Field „Name“: enter some name for the connection, e.g. WebGIS

Field "Host": 130.75.188.102

Field "Port": 5432

Field "Database": webgis

☒ only use schema 'public'

☒ estimate table metadata

Save all settings

Window "Data source management | PostgreSQL" (or similar)

select your previously created connection (e.g. WebGIS) from the choice widget
click Button "connect"

double-click the desired geometry table from the list below,

e.g. planet_osm_point, planet_osm_line, planet_osm_polygon

in the new dialogue, add filter statements into the text field (no "WHERE", just the conditions), e.g. "shop is not null", save settings

add layer

Task 5.5: (Mandatory for participants of final exercise) Assemble filter query for final exercise topic

Using the SQL filter/inspection commands from the previous tasks and the OSM feature list https://wiki.openstreetmap.org/wiki/Map_features, define the relevant objects from the database corresponding to your final exercise topic by identifying the filter criteria for each class of objects you want to display in your thematic map.

1. Begin by brainstorming relevant object classes in the context of your topic
2. Find out how these object classes are represented by OpenStreetMap, i.e. which key=value pairs are used to represent those objects in the database. In some cases, objects may require a combination of filter criteria. Take note of the commonly associated geometry type (single point, line/linestring, polygon) from column "Element" in the OSM feature list.

Check the final exercise database for occurrences of your selected objects, using the count(*) operator to quantify how many of these objects are present in the DB (= how many corresponding objects will appear on your map). Ignore object classes which are not represented well in the database (i.e. which are irrelevant for potential users).

```
SELECT count(*) from <table> WHERE <filter_criteria>;
```

3. Inspect the non-spatial properties in the table for each of your selected object classes. Which properties are usually associated with the object class? Are there any (computed) spatial properties that may be relevant in the context of your topic?
4. For the final exercise we expect a selection of multiple object classes that represent your topic well enough, both thematically as well as quantitatively (topics with low object density can compensate that by using more object classes).

Send a mail detailing your findings to colin.fischer@ikg.uni-hannover.de until **06.12.2022**. This includes your final exercise topics, the object classes you intend to display in your map, the filter criteria, number of objects and additional properties corresponding to each of those object classes. We will have a look at your selection of topics and give feedback on whether your choices are suitable (thematically and quantitatively) for the final exercise.